Exercice Python et SQL

Consultation des vols au Cameroun avec et sans jointure

Énoncé

L'agence de voyages souhaite consulter les données de sa base depuis un programme Python. Reprenez la base de l'exercice sur les aéroports du Cameroun et développez un programme qui affiche les aéroports, les vols et les passagers. Certaines consultations utiliseront une seule table ; d'autres relieront plusieurs tables avec des jointures.

Le programme doit être organisé en modules Python. Les requêtes SQL sont écrites dans un module dédié et leurs résultats sont affichés dans le terminal.

Préparation

Utilisez la base gestion_aeroports.db de l'exercice précédent. Si nécessaire, créez-la une seule fois en exécutant Exercice_Application_Aeroports_Cameroun.sql dans une nouvelle base SQLite. Elle contient 5 aéroports, 6 vols et 8 passagers fictifs.

Table	Colonnes
Aeroports	id_aeroport, nom, ville
Vols	id_vol, numero_vol, id_depart, id_arrivee, date_vol, statut
Passagers	id_passager, nom, prenom, email, id_vol

Liens : Vols.id_depart et Vols.id_arrivee référencent Aeroports.id_aeroport. Passagers.id_vol référence Vols.id_vol.

Organisation du programme

Placez les quatre fichiers Python et gestion_aeroports.db dans un même dossier. Respectez les responsabilités suivantes.

Module	Responsabilité
connexion.py	Définir ouvrir_connexion(chemin_db), qui ouvre la base existante et renvoie la connexion.
requetes.py	Définir une fonction par consultation. Chaque fonction reçoit la connexion, exécute sa requête et renvoie les lignes.
affichage.py	Définir afficher_resultats(titre, lignes), qui affiche un titre et les résultats de manière lisible.
main.py	Importer les fonctions, ouvrir une connexion, lancer les consultations et afficher leurs résultats, puis fermer la connexion.

Travail demandé

1 Mise en place des modules

1. Créer les quatre modules décrits à la page précédente. Utiliser le module sqlite3 de Python pour la connexion et l'exécution des requêtes.
2. Dans main.py, vérifier que le fichier de base existe avant de l'ouvrir. Fermer la connexion dans un bloc finally, y compris si une consultation échoue.

2 Affichages sans jointure

Écrire les fonctions suivantes dans requetes.py, puis les appeler depuis main.py. Chaque SELECT de cette partie porte sur une seule table, sans JOIN.

3. Afficher le nom et la ville de tous les aéroports, triés par ville.
4. Afficher le numéro, la date et le statut des vols dont le statut vaut Programme. Passer le statut en paramètre à la fonction.
5. Afficher le nom et le prénom des passagers du vol CM101 en deux étapes : rechercher son id_vol dans Vols, puis utiliser cet identifiant pour consulter Passagers. Passer les valeurs recherchées en paramètres.

3 Affichages avec jointures

Ajouter les fonctions suivantes dans requetes.py, puis afficher leurs résultats depuis main.py.

6. Afficher le nom et le prénom de chaque passager avec le numéro de son vol. Utiliser INNER JOIN entre Passagers et Vols.
7. Afficher chaque numéro de vol avec sa ville de départ et sa ville d'arrivée. Relier Vols deux fois à Aeroports, avec deux alias différents.
8. Afficher les passagers du vol CM101 avec une seule requête utilisant INNER JOIN. Comparer les noms obtenus avec ceux de la consigne 5.
9. Afficher tous les vols avec le nom et le prénom de leurs passagers, y compris les vols sans inscription. Utiliser LEFT JOIN en partant de Vols et afficher « Aucun passager » lorsque les données du passager valent None en Python.